HELM: Dynamic Semantic Routing for Heterogeneous Multi-Agent Systems

Vlad Digori
vdigori@lasige.di.fc.ul.pt
Faculdade de Ciências da Universidade de Lisboa
Lisbon, Portugal

Luís Pinto
lmpinto@ciencias.ulisboa.pt
Faculdade de Ciências da Universidade de Lisboa
Lisbon, Portugal

João Pedro Carvalho
jpecarvalho@ciencias.ulisboa.pt
Faculdade de Ciências da Universidade de Lisboa
Lisbon, Portugal

Abstract

Multi-agent systems composed of heterogeneous and dynamically available agents require an effective mechanism for determining which agents are suitable for a given task. Static routing mechanisms can limit adaptability, particularly when agents may join or leave during operation or expose overlapping specialized capabilities. This paper presents HELM, a multi-agent orchestration system that maintains a live semantic directory of agents communicating through IRC. Agents advertise natural-language capability descriptions that are dynamically indexed in ChromaDB and used for semantic retrieval. Two routing strategies are investigated: direct Top-1 semantic routing, in which the nearest retrieved agent is selected, and a hybrid Top-k strategy in which ChromaDB retrieves multiple candidate agents and an LLM performs the final selection. HELM is evaluated on 120 missions covering informational, planning, analytical, and robotic capabilities. The best Top-k configuration increases routing accuracy from 65.8% to 80.8% and final HELM answer accuracy from 59.2% to 76.7%. The results indicate that semantic capability descriptions provide a useful basis for dynamic agent discovery, while candidate-based LLM selection can reduce errors caused by selecting a single nearest semantic match.

Keywords

multi-agent systems, semantic routing, agent orchestration, large language models, vector databases, robotics, ChromaDB

1 Introduction

Multi-agent systems enable complex tasks to be distributed among specialized components with different capabilities. This becomes particularly relevant in robotic environments, where a mission may require information retrieval, planning, environmental analysis, decision making, and physical execution. Distributing these responsibilities among specialized agents allows a system to combine different forms of expertise while maintaining a common coordination mechanism.

As multi-agent systems grow in size and heterogeneity, determining which agent should handle a given request becomes increasingly challenging. A natural-language mission does not necessarily identify the required agent explicitly and may instead describe a goal that can be addressed by one or more specialized capabilities. For example, a mission involving a location may require information from a mapping agent, while a request involving physical movement may require a robotic actuator. The problem becomes more complex in dynamic environments, where agents may join or leave the system and where multiple agents can expose related but operationally different capabilities. Consequently, an orchestration mechanism must interpret an incoming mission and associate it with the capabilities of the agents that are currently available.

Existing approaches to multi-agent routing highlight the difficulty of coordinating heterogeneous agents efficiently. Recent work has explored learned routing mechanisms that jointly consider collaboration strategies, agent roles, and model selection [10], as well as state-aware approaches that adapt agent selection according to interaction history and available agent knowledge [7]. Other approaches investigate optimization mechanisms intended to improve routing efficiency and interpretability [8]. These methods demonstrate the importance of adaptive routing, but they may introduce additional learned controllers, routing models, or optimization mechanisms. This motivates investigating whether dynamically advertised capability descriptions can themselves provide an effective routing signal.

To explore this problem, this paper presents HELM, a multi-agent orchestration system based on dynamic capability discovery and semantic routing. Agents advertise natural-language descriptions of their capabilities while connected to the system, allowing HELM to maintain a live representation of the available agent population. Natural-language missions are matched against these descriptions using vector embeddings stored in ChromaDB. HELM supports two routing strategies. The first directly selects the closest semantic match, whereas the second retrieves several candidate agents and allows an LLM to select the best candidate according to the mission and the advertised capabilities.

Based on this architecture, this work investigates how effectively capability descriptions can support agent selection in a dynamic and heterogeneous multi-agent system and whether candidate-based LLM selection improves upon direct semantic routing. The evaluation therefore focuses on the following research question: *To what extent can semantic capability retrieval provide accurate routing in a dynamic multi-agent system, and how does selecting from multiple retrieved candidates using an LLM affect routing and end-to-end mission performance?*

The main contributions of this work are threefold. First, HELM provides a dynamic agent-discovery mechanism in which heterogeneous agents advertise their capabilities through live announcements. Second, it implements and compares two capability-based routing strategies: direct Top-1 semantic selection and Top-k semantic retrieval followed by LLM-based candidate selection. Third, the architecture integrates informational, planning, analytical, and robotic agents within a common orchestration framework. The two routing strategies are experimentally evaluated on the same set of 120 natural-language missions to analyze retrieval behaviour,

routing accuracy, answer correctness, and end-to-end system performance.

2 Background and Related Work

2.1 Multi-Agent Coordination and LLM Agents

Multi-agent systems distribute tasks among multiple specialized entities, making coordination and task allocation central problems when agents expose different capabilities. In heterogeneous systems, the selection process must account for which agents are capable of performing a given task and how their individual contributions can be combined [3, 5].

The emergence of large language models has extended this paradigm by enabling agents to interpret natural-language instructions, use external tools, and communicate with other agents. Frameworks such as AutoGen support conversational interaction between configurable LLM-based agents [9], while MetaGPT organizes agents into specialized roles connected through structured workflows [4]. AgentVerse similarly investigates collaborative groups of specialized agents across reasoning, tool-use, and embodied tasks [2].

These approaches demonstrate the potential of specialization and communication in LLM-based multi-agent systems. However, increasing the number and diversity of available agents also increases the difficulty of deciding which capability is relevant to each incoming request. HELM addresses this problem by treating agent selection as an explicit routing stage within the orchestration pipeline.

2.2 Semantic Routing and Agent Selection

Recent work has investigated adaptive mechanisms for selecting appropriate agents and collaboration strategies. MasRouter formulates routing as a learned decision process that considers collaboration mode, agent roles, and underlying language models [10]. State-aware approaches further incorporate information obtained during previous interactions when determining which agents should participate in subsequent steps [7]. Optimization-based approaches have also been proposed to improve the efficiency and interpretability of routing decisions [8]. These methods illustrate a broader transition from fixed routing policies toward mechanisms that adapt to the task and system state.

Sentence embedding methods provide another basis for routing by representing natural-language inputs as vectors whose relative positions capture aspects of semantic similarity. Sentence-BERT, for example, produces sentence-level representations that can be compared efficiently using similarity measures [6]. This makes embedding-based retrieval applicable to routing problems in which both requests and agent capabilities are expressed in natural language. Rather than requiring exact keyword correspondence, a mission can be associated with a capability whose description is semantically related to the requested task.

HELM investigates both direct semantic selection and a hybrid retrieval architecture. In the first case, semantic similarity determines the final selected agent. In the second, semantic retrieval is used to construct a small candidate set, after which an LLM performs a more detailed capability comparison. This provides a way to examine whether semantic retrieval alone is sufficient or whether a second-stage selector improves agent selection when capabilities overlap.

2.3 Multi-Agent Robotic Systems

Agent selection is also closely related to multi-robot task allocation, where tasks must be assigned according to the capabilities and constraints of available robots. Gerkey and Matarić formalized this problem and proposed a taxonomy for different forms of multi-robot task allocation [3]. Korsah et al. later extended this taxonomy to account for more complex tasks, heterogeneous teams, and interdependencies between assignments [5]. More recent reviews identify heterogeneity, dynamic environments, scalability, and task constraints as continuing challenges in multi-robot allocation [1].

HELM connects this task-allocation setting with a broader heterogeneous agent environment in which informational and physical capabilities coexist. A mission may require information from mapping, planning, or analytical agents, while other missions require physical actuation. This creates an orchestration problem in which the routing mechanism must distinguish between capabilities that may be semantically related but operationally different.

3 HELM Architecture and Routing

HELM is organized around a central orchestration component, referred to as the HELM Boss, a semantic directory backed by ChromaDB, and a set of heterogeneous agents communicating through an IRC channel. The architecture separates communication, capability discovery, semantic retrieval, candidate selection, agent execution, and final response generation.

3.1 System Architecture

A user submits a natural-language mission either through the shared IRC channel or directly to the HELM Boss. The Boss acts as the central mission coordinator. Instead of maintaining a fixed list of agent capabilities, it relies on a semantic directory that is constructed from live agent announcements.

The directory maintains information about agents currently connected to the system. Agents can belong to different capability groups, including mapping, planning, analysis, and robotic actuation. ChromaDB provides the vector-storage and retrieval layer used to index the natural-language capability descriptions.

The LLM service can be used at different stages depending on the routing configuration. In the Top-1 strategy, the final routing decision is performed directly by semantic retrieval. In the Top-k strategy, the LLM receives only the retrieved candidate descriptions and performs the final selection among those candidates.

The Boss is additionally responsible for sending requests to selected agents, collecting their replies, determining whether further interaction is required, and producing the final HELM response.

3.2 Dynamic Agent Discovery

HELM performs routing over a dynamically constructed catalog of currently available agents. Each agent joining the IRC channel announces information including its nickname, family, subfamily, and a natural-language capability description. Conceptually, an announcement has the following form:

```
nick=<agent>;
```

```
family=<family>;
subfamily=<subfamily>;
description=<capability>;
```

The semantic directory records these announcements and updates its indexed representation as agents join or leave the system. Consequently, the router does not depend on a fixed list of individual agents embedded directly in the orchestration logic.

Let the set of currently available agents be

$$A = \{a_1, a_2, \ldots, a_n\},$$

where each agent a_i provides a natural-language capability description c_i. Let $E(\cdot)$ denote the embedding function. For an incoming mission x, both the mission and agent descriptions are represented in the same vector space:

$$\mathbf{q} = E(x), \qquad \mathbf{v}_i = E(c_i).$$

The semantic distance between the mission and an agent capability is then

$$d\left(\mathbf{q}, \mathbf{v}_i\right).$$

Smaller distances indicate closer semantic matches between the mission and the advertised capability.

3.3 Routing Strategies

HELM was evaluated using two implementations of the agent-selection stage. Both strategies receive the same natural-language mission and operate over the same live capability directory maintained in ChromaDB. In both cases, agents advertise their capabilities through natural-language descriptions, and the incoming mission is embedded in the same vector space as those descriptions. The difference lies in how the final agent is selected after semantic retrieval.

Figure 1 illustrates the two approaches considered in this work. In the first approach, semantic retrieval directly determines the selected agent. In the second approach, semantic retrieval is used only to produce a small set of candidate agents, after which an LLM performs the final selection. This distinction makes it possible to compare whether embedding similarity alone is sufficient for routing, or whether a second reasoning stage improves performance when several agents have partially overlapping capabilities.

Approach 1: Top-1 Semantic Routing. The first routing strategy uses ChromaDB as the complete selection mechanism. Given a mission x, HELM embeds the mission and compares it against the capability descriptions of the currently available agents. The single closest agent is then selected directly:

$$a^* = \arg\min_{a_i \in A_x} d\left(E(x), E(c_i)\right),$$

where A_x is the set of agents eligible for the mission, $E(\cdot)$ denotes the embedding function, and $d(\cdot, \cdot)$ represents the semantic distance between the mission embedding and the capability embedding.

In this approach, the agent with the smallest semantic distance is assumed to be the best match for the mission. The main advantage of this strategy is its simplicity. It requires only a single retrieval step and introduces no additional reasoning layer at routing time. This makes the approach efficient and straightforward to implement.

However, Top-1 routing is also restrictive. Once the nearest match is selected, all other semantically similar candidates are discarded. This can lead to errors when multiple agents use related vocabulary in their capability descriptions but differ in their actual operational role. For example, several agents may refer to movement, navigation, routes, or coordinates, while only one of them is actually appropriate for the task. In such cases, the nearest embedding match may not correspond to the most suitable operational agent.

Approach 2: Top-k Retrieval with LLM Selection. The second routing strategy separates candidate retrieval from final agent selection. Instead of selecting only the nearest capability description, ChromaDB retrieves the k closest candidate agents:

$$C_k(x) = \mathrm{TopK}_{a_i \in A_x}\left(-d\left(E(x), E(c_i)\right)\right),$$

where $C_k(x)$ is the set of Top-k candidates returned for mission x.

The original mission and the retrieved candidate descriptions are then passed to an LLM, which performs the final agent choice:

$$a^* = L\left(x, C_k(x)\right),$$

where L denotes the LLM-based selection function.

This hybrid design assigns different roles to the two stages. ChromaDB acts as a fast semantic retrieval mechanism that narrows the search space to a small set of plausible candidates, while the LLM performs a more detailed comparison between the mission and the candidate capabilities. The LLM therefore does not search the full agent population. Instead, it reasons only over the retrieved candidate set and selects the one whose advertised capability best matches the mission requirements.

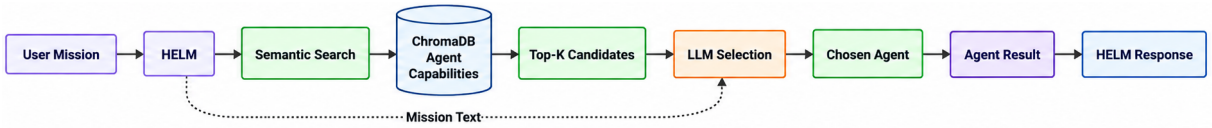
The motivation for this approach is that semantic similarity alone may not fully capture operational distinctions between agents. Two or more agents may appear close in embedding space because they share related concepts, while still differing in important constraints, accepted inputs, or intended behavior. By preserving several candidates instead of committing immediately to a single semantic match, the Top-k strategy gives the LLM an opportunity to resolve these ambiguities and choose the agent that is most appropriate in context.

Comparison of the Two Approaches. The two strategies differ only in the final selection stage, while sharing the same live directory, the same capability representations, and the same execution pipeline. This makes them directly comparable. Approach 1 tests whether semantic retrieval alone is sufficient for accurate routing. In contrast, Approach 2 tests whether a candidate-based LLM selector can improve routing performance by reasoning over a restricted set of semantically relevant alternatives.

From an evaluation perspective, this distinction is also important when analysing system errors. In the Top-1 approach, retrieval and routing are the same decision, since the retrieved top result is the selected agent. In the Top-k approach, these stages can be separated: the expected agent may be successfully retrieved among the candidates but not selected by the LLM. This allows failures to



(a) Approach 1: direct Top-1 semantic routing. The mission is embedded and compared with the live agent capability descriptions. ChromaDB returns the nearest semantic match, which is directly selected and invoked by HELM.



(b) Approach 2: Top-k semantic retrieval followed by LLM-based selection. ChromaDB first retrieves multiple candidate agents. The original mission and candidate capability descriptions are then given to an LLM, which selects the agent to invoke.

Figure 1: The two routing strategies evaluated in HELM. Approach 1 uses semantic retrieval as the final routing decision, while Approach 2 separates candidate retrieval from final agent selection by introducing an LLM-based selection stage.

be attributed more precisely either to candidate retrieval or to the final routing decision.

3.4 Mission Orchestration

After an agent is selected, the HELM Boss communicates with it through IRC and collects the response. The response can either directly answer the user's request or contribute information required by later mission stages.

Listing 1 shows a simplified example of the IRC message exchange during a Top-k mission. The example illustrates candidate retrieval, LLM-based selection, specialist execution, and the final response returned by HELM.

Listing 1: Simplified IRC interaction during a HELM mission.

```
User: HELM: Where are the red cars?

HELM -> ChromaDB:
Retrieve Top-5 candidate agents

ChromaDB -> HELM:
fig, grape, kiwi, plum, melon

HELM -> LLM Selector:
Mission: Where are the red cars?
Candidates: fig, grape, kiwi, plum, melon

LLM Selector -> HELM:
Selected agent: grape

HELM -> grape:
Where are the red cars?

grape -> HELM:
Red cars are located in zone B2.

HELM -> User:
The red cars are located in zone B2.
```

Some requests require additional processing beyond agent selection. HELM can collect contextual information from agents and use the result to construct a final user-facing answer. In missions involving physical actuation, collected context may also be transformed into an instruction for an eligible robotic agent.

The final HELM response is therefore evaluated separately from the answer produced directly by the selected specialist. This distinction makes it possible to identify whether a mission failed because the wrong agent was selected, because the specialist produced an incorrect answer, or because the final orchestration stage lost or incorrectly reformulated information.

Figure 2 summarizes the routing and execution process.

4 Evaluation

The evaluation compares two routing strategies across four configurations using the same set of natural-language missions and expected target agents. The first strategy uses direct Top-1 semantic retrieval, while the second uses Top-5 semantic retrieval followed by LLM-based candidate selection with three different selector models.

4.1 Experimental Setup and Test Missions

The evaluation was conducted using a fixed set of 120 natural-language missions distributed across 12 specialist agents, with 10 missions associated with each expected agent.

Model Configuration. ChromaDB was used as the semantic retrieval layer, with `qwen3-embedding:0.6b` generating embeddings for missions and agent capability descriptions. The specialist agents used `gpt-oss:120b-cloud` to generate their task responses throughout all experiments. These components were kept fixed so that differences between runs were primarily attributable to the routing configuration.

Routing Configurations. The first configuration, denoted R1, represents the retrieval-only baseline. ChromaDB directly returned the single nearest agent ($k = 1$), and no LLM was used for candidate selection.

The remaining configurations used Top-5 semantic retrieval ($k = 5$). ChromaDB first returned the five nearest candidate agents, after which an LLM selected the most appropriate candidate. R2 used `gemma4:31b-cloud`, R3 used `gpt-oss:120b-cloud`, and R4 used `nemotron-3-super:cloud` as the candidate-selection model.

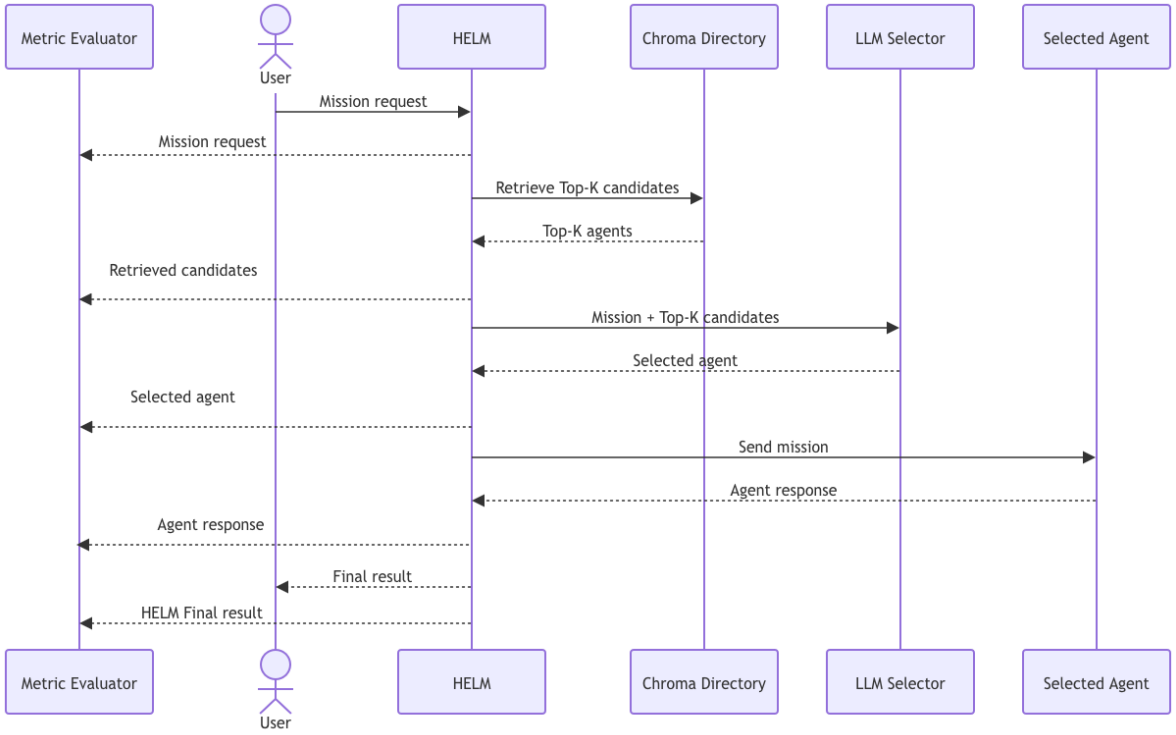


Figure 2: Mission routing and execution flow in HELM. An incoming mission is processed by the semantic routing stage, which either directly selects the Top-1 agent or retrieves a Top-k candidate set for LLM-based selection. The selected specialist receives the mission, produces a response, and returns it to the HELM Boss, which constructs the final user-facing response.

Table 1: Routing configurations used in the evaluation.

ID	Retrieval	Selector	k
R1	ChromaDB	None	1
R2	ChromaDB	Gemma4 31B	5
R3	ChromaDB	GPT-OSS 120B	5
R4	ChromaDB	Nemotron-3 Super	5

4.2 Evaluation Metrics

Several metrics are used to separate the different stages of the routing and orchestration pipeline.

Retrieval recall. Retrieval recall measures whether the expected agent is present in the candidate set produced by semantic retrieval:

$$\text{Retrieval Recall} = \frac{N_{\text{expected agent retrieved}}}{N_{\text{missions}}}.$$

For Top-1 routing, the candidate set contains only one agent. Therefore, retrieval recall and routing accuracy are equivalent for this strategy.

Routing accuracy. Routing accuracy measures how frequently the expected agent is selected:

$$\text{Routing Accuracy} = \frac{N_{\text{correct routes}}}{N_{\text{missions}}}.$$

Only expected agent called. This metric measures the proportion of missions in which only the expected specialist was invoked:

$$\text{Expected-Only Rate} = \frac{N_{\text{only expected agent called}}}{N_{\text{missions}}}.$$

Agent answer accuracy. Agent answer accuracy measures whether the expected specialist provided the required answer across the complete evaluation set:

$$\text{Agent Answer Accuracy} = \frac{N_{\text{correct agent answers}}}{N_{\text{missions}}}.$$

Answer accuracy given called. This metric isolates specialist behaviour from routing behaviour:

$$\text{Answer Accuracy Given Called} = \frac{N_{\text{correct answers from expected agent}}}{N_{\text{missions where expected agent was called}}}.$$

HELM final-answer accuracy. Finally, HELM final-answer accuracy measures whether the final response returned by the orchestration system contains the required answer:

$$\text{HELM Answer Accuracy} = \frac{N_{\text{correct HELM responses}}}{N_{\text{missions}}}.$$

4.3 Routing Strategy Comparison

Table 2 compares the four evaluated routing configurations.

The retrieval-only Top-1 baseline achieved a routing accuracy of 65.8%. All three Top-5 configurations improved upon this result. Gemma4 and Nemotron both reached 80.8% routing accuracy, while GPT-OSS achieved 77.5%.

The common retrieval recall of 81.7% across all Top-5 configurations shows that the candidate-generation stage was identical and that the differences between the three hybrid configurations were introduced by the LLM-based selection and orchestration stages. Compared with the Top-1 baseline, the Top-5 candidate set increased the probability that the expected agent remained available for final selection.

The largest difference between selector models appears in the final HELM response. Gemma4 achieved the highest final-answer accuracy at 76.7%, followed by Nemotron at 72.5%. GPT-OSS obtained only 51.7%, despite reaching 77.5% routing accuracy and 71.7% agent-answer accuracy. This indicates that correct routing alone does not guarantee a correct final system response and that the model used during orchestration can substantially affect how agent outputs are summarized or transformed.

The results also show that the nominal size of the selector model did not directly correspond to routing accuracy. Gemma4 31B matched the 80.8% routing accuracy obtained with Nemotron-3 Super and exceeded the 77.5% obtained with GPT-OSS 120B. Within this evaluation, selector performance therefore appears to depend on model behavior and the interaction with the selection prompt, rather than on model scale alone.

Figures 3 and 4 provide a stage-by-stage failure decomposition of the complete HELM evaluation pipeline. Each mission begins in the total set of 120 missions and continues along the main flow only while the expected behaviour is preserved. When a mission fails a particular evaluation condition, it leaves the main path through the corresponding red branch.

The first stage evaluates semantic retrieval. A mission is counted as *Retrieved* when the expected agent is contained in the candidate set returned by ChromaDB. For R1, the candidate set contains only the Top-1 result. Consequently, retrieval and routing are equivalent because the retrieved agent is immediately selected. The 41 missions shown as *Retrieval failed* therefore correspond to cases where the expected agent was not the nearest semantic match.

For R2–R4, retrieval and routing are evaluated separately. ChromaDB first returns the five nearest candidates. A mission reaches the *Correctly routed* stage only when the LLM subsequently selects the expected agent from this candidate set. A *Retrieval failed* branch therefore means that the correct agent was absent from the Top-5 candidates, while a *Routing failed* branch means that the expected agent was available to the selector but the LLM chose another candidate.

The next stage evaluates the specialist response. A mission reaches *Correct agent answer* when the selected expected agent returns the required ground-truth information. The *Agent answer failed* branch contains missions where routing was correct but the expected specialist's response did not satisfy the benchmark answer criteria.

The final stage evaluates the response produced by HELM after the specialist reply has been received. A mission is counted as *Successful* only when the final user-facing HELM response still contains the required information. The *HELM final failed* branch therefore represents missions where the specialist produced a correct answer, but the final orchestration response lost, changed, or failed to preserve that information.

The Sankey diagrams therefore separate failures according to the stage at which they occur: semantic retrieval, LLM-based routing, specialist answering, or final response generation. This makes it possible to distinguish routing problems from downstream answer-generation problems rather than representing all unsuccessful missions as a single error category.

5 Discussion and Limitations

The experimental results show that retrieving multiple semantic candidates before the final routing decision can improve overall routing performance. Direct Top-1 retrieval correctly selected the expected agent in 65.8% of the evaluated missions, whereas Top-k retrieval followed by LLM selection reached 80.8%. This suggests that selecting only the nearest embedding result can discard useful alternatives when capabilities occupy similar regions of the semantic space.

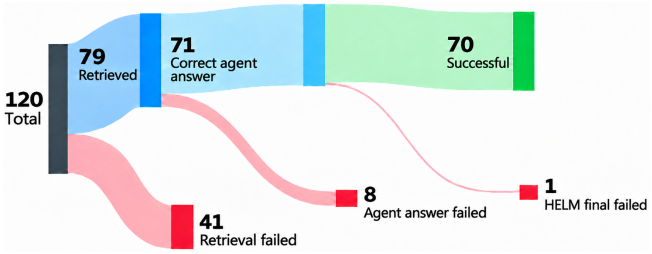
The distinction between retrieval recall and routing accuracy is important for interpreting the hybrid approach. The expected agent appeared in the Top-5 candidate set in 81.7% of missions for all three LLM-based configurations. Final routing accuracy ranged from 77.5% to 80.8%, indicating that most routing errors in the hybrid configurations originated either from the retrieval stage or from a relatively small number of incorrect selections within the retrieved candidate set. Because the selector cannot choose an agent absent from the candidate set, retrieval recall establishes an upper bound on routing accuracy for this architecture. When semantic retrieval failed to include the expected capability, the second stage could not recover from the error. Candidate retrieval therefore remains an important limiting factor.

Another limitation is capability overlap. Several robotic agents share semantically similar descriptions but operate under different constraints. Embedding models can recognize that all of these agents are related to movement or navigation without necessarily encoding the exact operational distinction required by a particular request. The strong improvement for some agents, such as cherry and date, indicates that the second-stage selector helps with this problem, but the poor banana results show that it is not fully resolved.

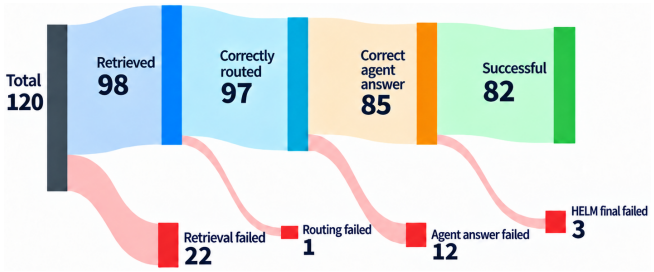
Another limitation of the evaluation is that HELM was not tested as a standalone routing mechanism. The experiments compared direct ChromaDB Top-1 retrieval with hybrid configurations in which ChromaDB retrieved a Top-k candidate set and HELM used an LLM to perform the final selection. However, a configuration in which HELM alone selected an agent without semantic retrieval was not evaluated. As a result, the current experiments demonstrate the effect of combining semantic retrieval with LLM-based selection, but they do not isolate the performance of an LLM-only routing configuration. Including such a baseline would provide a clearer

Table 2: Comparison of HELM routing configurations on 120 missions.

Metric	R1	R2	R3	R4
Retrieval Recall	65.8%	81.7%	81.7%	81.7%
Routing Accuracy	65.8%	80.8%	77.5%	80.8%
Only Expected Agent Called	65.8%	71.7%	60.0%	73.3%
Answer Accuracy Given Called	89.9%	87.6%	92.5%	90.7%
Agent Answer Accuracy	59.2%	70.8%	71.7%	73.3%
HELM Final-Answer Accuracy	59.2%	76.7%	51.7%	72.5%

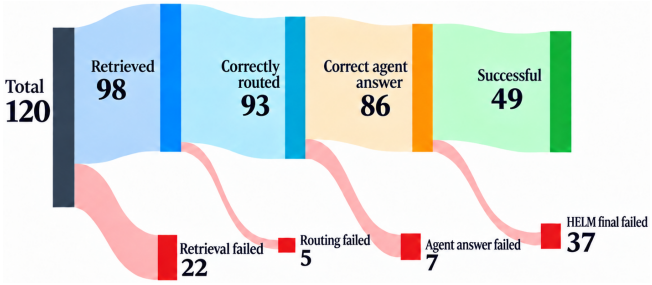


(a) R1: Top-1 semantic routing. ChromaDB directly selects the nearest agent.

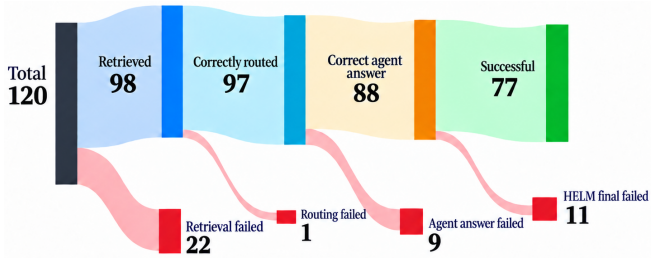


(b) R2: Top-5 semantic retrieval followed by Gemma4 31B candidate selection.

Figure 3: Stage-by-stage evaluation funnels for R1 and R2. The main horizontal flow represents missions that remain correct through successive stages, while red branches indicate the stage at which missions fail.



(a) R3: Top-5 semantic retrieval followed by GPT-OSS 120B candidate selection.



(b) R4: Top-5 semantic retrieval followed by Nemotron-3 Super candidate selection.

Figure 4: Stage-by-stage evaluation funnels for R3 and R4. "Retrieved" indicates that the expected agent was present in the semantic candidate set; "Correctly routed" indicates that HELM selected the expected agent; "Correct agent answer" indicates that the expected specialist returned the required answer; and "Successful" indicates that the final HELM response also preserved the required information.

comparison between embedding-based retrieval, LLM-only routing, and the hybrid approach.

Finally, the current benchmark contains 12 agents and 120 missions in a controlled environment. Although the agent descriptions represent heterogeneous capabilities, these results do not establish how the routing strategies would behave with hundreds of dynamically changing agents or with many highly overlapping descriptions. Larger evaluations are therefore required to assess scalability and robustness.

6 Conclusion and Future Work

This paper presented HELM, a multi-agent orchestration system designed to route natural-language missions among dynamically available heterogeneous agents. Agents advertise their capabilities through live announcements, and these descriptions are embedded and indexed in ChromaDB to create a semantic representation of the capabilities currently available in the system.

Two routing strategies were evaluated using the same set of 120 missions. Direct Top-1 semantic routing selected the nearest capability and achieved 65.8% routing accuracy. Two routing strategies were evaluated using the same set of 120 missions. Direct Top-1

semantic routing achieved 65.8% routing accuracy. The hybrid strategy retrieved the five nearest semantic candidates and used one of three LLMs for final selection. Gemma4 31B and Nemotron-3 Super both achieved 80.8% routing accuracy, while GPT-OSS 120B achieved 77.5%. The selector models differed more substantially in later orchestration stages, with final HELM answer accuracy ranging from 51.7% to 76.7%. These results indicate that Top-k retrieval can improve agent selection over direct nearest-neighbor routing, while the choice of selector model also influences downstream system behaviour. HELM final-answer accuracy also increased from 59.2% to 76.7%. These results suggest that semantic retrieval provides a useful basis for dynamic agent discovery, while a candidate-selection stage can improve routing when multiple capabilities are semantically similar.

The experiments also reveal several limitations. Some capabilities remain difficult to distinguish, particularly among robotic agents with related movement terminology. Furthermore, the hybrid strategy depends on the expected agent being present in the retrieved candidate set and introduces additional LLM inference cost.

Future work will investigate the influence of different values of k, routing confidence thresholds, improved capability descriptions, alternative reranking methods, and automatic capability learning. Additional experiments should also measure latency and computational cost, evaluate larger agent populations, and investigate real robotic deployments. A further direction is the development of explainable routing traces that expose both semantic retrieval scores and the reasoning behind the final candidate-selection decision.

Ethics and Privacy Statement

HELM can produce instructions that are eventually delivered to physical robotic agents. Real deployments should therefore apply appropriate operational constraints and human supervision before executing safety-critical actions. Information exchanged between agents should also be handled according to the privacy requirements of the deployment environment.

References

[1] Hamza Chakraa, François Guérin, Edouard Leclercq, and Dimitri Lefebvre. 2023. Optimization Techniques for Multi-Robot Task Allocation Problems: Review on the State-of-the-Art. *Robotics and Autonomous Systems* 168 (2023), 104492. doi:10.1016/j.robot.2023.104492

[2] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *International Conference on Learning Representations*.

[3] Brian P. Gerkey and Maja J. Matarić. 2004. A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems. *The International Journal of Robotics Research* 23, 9 (2004), 939–954. doi:10.1177/0278364904045564

[4] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*.

[5] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. 2013. A Comprehensive Taxonomy for Multi-Robot Task Allocation. *The International Journal of Robotics Research* 32, 12 (2013). doi:10.1177/0278364913496484

[6] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 3982–3992. doi:10.18653/v1/D19-1410

[7] Jingbo Wang, Sendong Zhao, Haochun Wang, Yuzheng Fan, Lizhe Zhang, Yan Liu, and Ting Liu. 2025. Optimal-Agent-Selection: State-Aware Routing Framework for Efficient Multi-Agent Collaboration. arXiv:2511.02200 [cs.AI]

[8] Xudong Wang, Chaoning Zhang, Jiaquan Zhang, Chenghao Li, Qigan Sun, Sung-Ho Bae, Peng Wang, Ning Xie, Jie Zou, Yang Yang, and Hengtao Shen. 2026. Efficient and Interpretable Multi-Agent LLM Routing via Ant Colony Optimization. arXiv:2603.12933 [cs.AI]

[9] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *ICLR 2024 Workshop*.

[10] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. 2025. MasRouter: Learning to Route LLMs for Multi-Agent Systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 15549–15572. doi:10.18653/v1/2025.acl-long.757